Learn data skills 10x faster
Join 1M+ learners
Email address
Password
Start Now
Enroll for free
Data Analyst (Python)
Gen AI (Python)
SQL
Data Literacy using Excel
Business Analyst (Power BI)
Business Analyst (Tableau)
Machine Learning
Data Analyst (R)

Published: September 14, 2022

How to Copy a List in Python (5 Techniques w/ Examples)

Lists are commonly used data structures in Python. We will often encounter situations wherein we need to make a copy of a list, and you may ask yourself, "How can I copy a list in Python?" or "Which copy method suits my requirements best?"

This tutorial will teach you how to copy or clone a list using several different techniques:

- The assignment operator
- The slicing syntax
- The list.copy() method
- The copy.copy() function
- The copy.deepcopy() function

We will also discuss their usage and technical aspects in detail.

Copy a List Using the Assignment Operator

Suppose you use the assignment operator (=) to copy a list by assigning an existing list variable to a new list variable. In this case, you're not actually creating a copy of the list; you're just creating an alias that points to the exact same location in memory where the original list object exists. Let's expand on the details and look closer.

Suppose we have the list variable, `org_list`, defined as follows:

```
org_list = [1, 2, ['a', 'b', 'c'], 3]
```

Then, we assign it to a new variable, `cpy_list`, hoping to make a copy of it for future use:

```
cpy_list = org_list
```

However, you need to know the variable `cpy_list` isn't a true copy of the original list. You may ask, "Why isn't it a true copy of the original list?" This is a great question because, as you'll see below, printing these two variables returns the exact the same values.

```
print('Original List:', org_list)
print('Copied List:', cpy_list)
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3]
Copied List: [1, 2, ['a', 'b', 'c'], 3]
```

As expected, the lists contain the same values. But, let's see what happens if we modify the original list.

```
org_list.append('Dataquest')

print('Original List:', org_list)
print('Copied List:', cpy_list)
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3, 'Dataquest']
Copied List: [1, 2, ['a', 'b', 'c'], 3, 'Dataquest']
```

Any modification to the original list will change the copied list, too.

The following illustration shows what's happening once the source code is executed.

In fact, when you assign one variable to another, both variables are referencing the same object in memory, not two separate ones. This means both variables point to the same object via their references. When more than one variable references the same object, it's called a shared reference or object.

Any modification to a shared mutable object via one of the variables that points to it affects the other variable that references the same object.

So, using the assignment operator doesn't make a true copy of a list; it just creates an alias for the same object in memory.

But what if we want to make an independent copy of a list? In the following section, we'll learn how to make shallow copies of a list.

The Shallow Copy Techniques

We just learned that assignments always store references to objects and don't make an actual copy of those objects. However, it's essential to know that changing a **mutable** object affects other objects that use the same reference in our code. So, we need to let Python know explicitly to copy an object if we want more than just a copy of the reference to that object. Generally speaking, there are two ways of making an independent copy of a list: a shallow copy and a deep copy. This section will discuss shallow copy and the different ways of implementing it.

Simply put, making a shallow copy of a compound list creates a new compound list and uses the references to the objects that the original list used.

NOTE: A compound object is an object that contains other objects, e.g., lists or dictionaries.

Shallow Copy Using List Slicing

To understand the shallow copy concept, let's begin with an example. Assume we have a compound list as follows:

```
org_list = [1, 2, ['a', 'b', 'c'], 3]
```

Then, we can create a shallow copy of it using list slicing syntax:

```
cpy_list = org_list[:]
```

If we run the following print statements, we can see, both return exactly the same values.

```
print('Original List:', org_list)
print('Copied List:', cpy_list)
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3]
Copied List: [1, 2, ['a', 'b', 'c'], 3]
```

Now, let's append a new item to the original list and run the print statements again:

```
org_list.append('Dataquest')

print('Original List:', org_list)
print('Copied List:', cpy_list)
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3, 'Dataquest']
Copied List: [1, 2, ['a', 'b', 'c'], 3]
```

The modification doesn't affect the copied list. But, this isn't the whole story. Let's try another scenario and change one of the items in the nested list to see what happens:

```
org_list[2][0] = 'A'

print('Original List:', org_list)
print('Copied List:', cpy_list)
```

```
Original List: [1, 2, ['A', 'b', 'c'], 3, 'Dataquest']
Copied List: [1, 2, ['A', 'b', 'c'], 3]
```

Although making a shallow copy of a list produces a true copy of the original list, any modifications to the nested elements within it will be reflected in both lists. The reason is that the nested list in the copied list uses the same shared reference as the one in the original list. In other words, the nested lists in the copied list are tied to the nested lists in the original list. This is why we call it a shallow copy — because only a new top-level object is created while anything deeper uses a shared reference with the original list.

Now, let's look at some other ways of making shallow copies of a list.

The Python `list.copy()` Method

Earlier, we discussed creating a shallow copy via the slicing syntax. In this section, we'll learn about a built-in method that Python programmers commonly use for copying lists. The Python `copy()` method returns a shallow copy of the list without taking any parameters. Let's try it out:

```
org_list = [1, 2, ['a', 'b', 'c'], 3]

cpy_list = org_list.copy()

print('Original List:', org_list, ' @', id(org_list))
print('Copied List:', cpy_list, ' @', id(cpy_list))
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3] @ 140532204957920
Copied List:  [1, 2, ['a', 'b', 'c'], 3] @ 140532204957600
```

Although `org_list` and `cpy_list` have the same values, as the output from the `id()` function shows, they end up in different locations in memory. However, exposing the memory addresses of the inner lists in both the original and copied lists reveals that they refer to the same location in memory, meaning we made a shallow copy of the original list.

```
org_list[2][0] = 'A'

print('Inner List in Original: ', org_list[2], ' @', id(org_list[2]))
print('Inner List in Shallow Copied: ', cpy_list[2], ' @', id(cpy_list[2]))
```

```
Inner List in Original:  ['A', 'b', 'c'] @ 140532024059328
Inner List in Shallow Copied:  ['A', 'b', 'c'] @ 140532024059328
```

The Python `copy.copy()` Function

The other useful way of making a shallow copy of a list is the `copy.copy()` function. To use it, we import the `copy` module and then pass the list we want to copy to the `copy.copy()` function. Let's try it out:

```
import copy

org_list = [1, 2, ['a', 'b', 'c'], 3]

cpy_list = copy.copy(org_list)

print('Original List:', org_list, ' @', id(org_list))
print('Copied List:', cpy_list, ' @', id(cpy_list))
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3] @ 140532024760320
Copied List:  [1, 2, ['a', 'b', 'c'], 3] @ 140532024402640
```

Now, let's append a new item to the original list, print both lists again, and check the output; nevertheless, we can predict the output before running the code below.

```
org_list.append('Dataquest')

print('Original List:', org_list, ' @', id(org_list))
print('Copied List:', cpy_list, ' @', id(cpy_list))
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3, 'Dataquest'] @ 140532024760320
Copied List:  [1, 2, ['a', 'b', 'c'], 3] @ 140532024402640
```

The `copy.copy()` method has made a true copy of the original list. However, it's still a shallow copy, and the nested lists refer to exactly the same memory location. In other words, the `copy.copy()` function only makes top-level copies and doesn't copy nested objects. So, any modifications in either the original or copied list's nested objects reflects in the other list's nested objects.

```
org_list[2][0] = 'A'

print('Inner List in Original: ', org_list[2], ' @', id(org_list[2]))
print('Inner List in Shallow Copied: ', cpy_list[2], ' @', id(cpy_list[2]))
```

```
Inner List in Original:  ['A', 'b', 'c'] @ 140532024760128
Inner List in Shallow Copied:  ['A', 'b', 'c'] @ 140532024760128
```

What if we want a fully independent copy of a deeply nested list? In the next section, we'll discuss how to perform a deep copy in Python.

The Python `copy.deepcopy()` Function

The `copy.deepcopy()` function recursively traverses a list to make copies of each of its nested objects. In other words, it makes a top-level copy of a list and then recursively adds copies of the nested objects from the original list into the new copy. This produces a fully independent copy from the original list, and any changes made to the nested objects of either will not be reflected in the other.

Like the `copy.copy()` function, the `copy.deepcopy()` function belongs to the `copy` module. Let's try it out:

```
import copy

org_list = [1, 2, ['a', 'b', 'c'], 3]

cpy_list = copy.deepcopy(org_list)

print('Original List:', org_list, ' @', id(org_list))
print('Copied List:', cpy_list, ' @', id(cpy_list))
```

```
Original List: [1, 2, ['a', 'b', 'c'], 3] @ 140532024761480
Copied List:  [1, 2, ['a', 'b', 'c'], 3] @ 140532024405760
```

The output of the code above clearly shows that `copy.deepcopy()` has made a true copy of the original list, and even if we modify the inner list of the original list, it won't be reflected in the deep copied list.

```
org_list[2][0] = 'A'

print('Inner List in Original: ', org_list[2], ' @', id(org_list[2]))
print('Inner List in Deep Copied: ', cpy_list[2], ' @', id(cpy_list[2]))
```

```
Inner List in Original:  ['A', 'b', 'c'] @ 140532024404736
Inner List in Deep Copied:  ['A', 'b', 'c'] @ 140532024405248
```

The code above shows that when we create a deep copy of a list, it also makes true copies of the nested objects. As mentioned earlier, the recursive deep copy produces a truly independent copy of the original list, which is why the inner lists in the original and copied lists point to two different memory locations. Obviously, any changes made to the inner list of one won't be reflected in the other.

Conclusion

This tutorial discussed several different ways for copying a list in Python, such as the assignment operator, list slicing syntax, `list.copy()`, `copy.copy()`, and `copy.deepcopy()` functions. Also, we discussed shallow and deep copies. I hope this tutorial helps you to better understand the different ways of copying lists in Python because they are critical to becoming a Pythonista.

About the author

Mehdi Lotfinejad

Mehdi is a Senior Data Engineer and Team Lead at ADA. He is a professional trainer who loves writing data analytics tutorials.

More learning resources

Tutorial: Time Series Analysis with Pandas
Read more →

Pandas Tutorial: Analyzing Video Game Data with Python and Pandas
Read more →